Porting a continuous-energy Monte Carlo to CUDA in pure Rust:

What we built, what it weighs, and what we believed that wasn't true

Fábio Rodrigues
sorcerer86pt@gmail.com

2026

Abstract

We report the engineering pace and honest performance findings of porting `open_rust_mc` — a pure-Rust continuous-energy neutron / photon Monte Carlo transport code with recursive-CSG geometry, SVD-compressed cross sections, and ICSBEP regression substrate — to NVIDIA CUDA via the `cudarc` crate and NVRTC. The CPU and GPU backends share a single `EigenvalueRunner` trait so any benchmark runs through either path with the same scene JSON, the same nuclear data, and the same acceptance envelope.

The framing of this paper is not "what we discovered". The Monte Carlo XS-lookup memory-bandwidth regime is documented in Tramm & Siegel 2014; the GPU saturation curve and refill-pool optimisation are in the PHYSOR 2022 / EPJ 2024 line of work from the OpenMC group; the per-warp shared-memory (n, E_{idx}) cache idea has been a published optimisation for years. We hit these problems in roughly the order the literature predicts; we just took our time hitting them. The interesting content is the *pace* and the *order*: where the per-XS-lookup microbench misled us into expecting an integration win, how long it took to notice that the S(α,β) upload was the missing peer of the per-nuclide cache, why a "GPU is 6.74 ×" headline was carried across sessions without its scope tag of "const-XS geometry-traversal microbench", how many sessions were spent reading code to convince ourselves the cache fired before we added the observability path that just answered the question.

What landed in this round: a process-wide per-nuclide device-buffer cache keyed on `Arc::as_ptr`, a parallel cache for S(α,β) thermal-scattering payloads, adaptive NVRTC arch detection so one binary targets RTX A1000 through Blackwell RTX 5090 without rebuilds, the PHYSOR Optimization F refill pool with a device-attribute-driven auto-recommender, and a `--debug-metrics` JSON-lines path that snapshots cache entries, hit counts, byte usage, and the detected compute capability per case. The CPU backend on a 20-thread laptop still beats the GPU SVD path on Godiva and is competitive through PWR; the GPU wins start showing up where the literature said they would — 10^6+-history photon shielding and multi-case ICSBEP sweeps where rayon's cache contention dominates the host.

Code: github.com/sorcerer86pt/open_rust_mc

1 Introduction

This paper is a companion to the SVD cross-section compression paper. The headline of that paper is a physics-and-correctness result; this one is the engineering note that goes with it. After we shipped GPU backends for SVD reconstruction, recursive-CSG geometry, photon transport, and a multigroup random-ray solver, we found ourselves re-learning a small set of lessons that the literature broadcasts but that one only believes after living through them:

1. Kernel-level optimisation matters less than data-flow amortisation. A per-XS-lookup speedup of 8–13 × on the SVD reconstruction microbench does not survive full transport on a 9-nuclide PWR pin cell; what does survive is whether the per-nuclide payloads (hundreds of MB, sometimes GB on actinide-heavy cases) are uploaded once or once per case.

2. "GPU is faster than CPU" is a measurement-specific claim, not an architectural one. Our recursive-transport kernel runs 6.74 × faster than CPU *on the constant-XS geometry-traversal microbench*; on full Godiva SVD transport with 20 rayon worker threads on a workstation Intel mobile, the same GPU kernel is 1.3 × *slower*.

3. Observability needs to be in the loop. We spent meaningful sessions trying to confirm by reading stdout that a cache "actually fired". The fix was a ∼200-line debug-metrics path that exposes cache entry counts, hit totals, byte usage, and the detected compute capability via PyO3, with a single CLI flag on the ICSBEP sweep driver that dumps one JSON line per case. Per-case deltas in the hit counter make cache behaviour falsifiable.

The code is written in pure Rust against `cudarc` for the CUDA driver and runtime APIs, and against NVRTC for at-startup PTX compilation. There is no C dependency on the host side either — `hdf5-pure` reads the ENDF/B nuclear data directly, and the same `Scene` JSON corpus drives both backends.

Why CUDA at all. The SVD compression paper's PWR result already shows that GPU SVD is slower than CPU SVD on this code base for a 9-nuclide pin cell. We continued GPU work anyway for three reasons. First, problem-class scaling: ICSBEP cases reach ∼70 nuclides (HMF-069) and 17×17 PWR assembly at depth 3 has ∼300 universe cells; rayon's cache pressure on a 20-thread laptop becomes the dominant cost long before algorithmic complexity does. Second, hardware extrapolation: an RTX 3080 at saturation reaches 1.2 M histories/s on Godiva (measured, §5); an A100 / H100 extrapolates linearly with SM count and VRAM, and the same binary now targets all of them. Third, photon shielding

via persistent-kernel Compton walks does survive integration (2.22 × over 20-thread rayon CPU on 10^6 histories) — the photon side of the engine makes the GPU investment pay off cleanly.

What this paper covers. §2 describes the shared dispatch trait that lets CPU and GPU paths run the same scene. §3 is the engineering core: per-nuclide kernels, the new S(α,β) buffer cache, the `Arc::as_ptr` trick that makes both work, and what they weigh in practice. §4 catalogues the SM occupancy and register-pressure findings, including the `nuc_t[128]` per-thread stack array that pins our PTX to spillage on small cards. §5 reports the PHYSOR 2022 Optimization F refill-pool work with measured saturation curves on RTX A1000 / RTX 3080. §6 describes the runtime compute-capability detection that ended the "rebuild-per-card" tax. §7 introduces `--debug-metrics`. §8 consolidates the "what we thought vs. reality" findings; §9 closes.

What this paper does *not* cover. The kernel-level SVD reconstruction algorithm and its arithmetic-intensity tradeoffs are documented in the companion SVD paper. Algorithm-level correctness against ICSBEP handbook values, OpenMC cross-validation, and the ENDF/B-VII.1 vs. VIII.1 library transition are documented in the project's `STATUS.md` and benchmark CSVs; we cite a few headline numbers but do not re-derive them here.

2 Architecture

The CPU backend is history-based and rayon-parallel: each worker thread pulls a fresh particle from a shared source bank, walks it to death, accumulates per-batch tallies in a worker-local sink, and publishes the sinks back to the eigenvalue driver via a `fold().reduce()` reduction. The GPU backend is event-batched: particles live as Structures-of-Arrays in device memory, advance through a fixed pipeline of kernels (`find_cell_batch`, `trace_step_batch`, `multi_step_walk`, then the collision-sampling stages), and write tallies via `atomicAdd(double*, double)` on Ampere-or-newer hardware.

Both paths consume the same data structures upstream of the runner: the recursive `Geometry` (universes, hexagonal and rectangular lattices, BVH per universe), the `XsProvider` trait (one of `Table`, `Svd`, `HybridSvdWmp`, `HybridTableWmp`), the `ThermalScatteringData` registry, and the per-material atom-density tables. They diverge at the `EigenvalueRunner` trait, which has two implementations: `CpuRunner` (rayon) and `CudaRunner` (cudarc + NVRTC).

The shared trait. `EigenvalueRunner` exposes a single method `run(&self, config: SimConfig) -> OutcomeResult`. Every benchmark, every test, and every Python entry point routes through this method; the choice of backend is a runtime variable, and the scene JSON corpus does not branch on it. This is how we run the same 7-case heu-comp-inter-003 family through CPU and GPU back-to-back with one CLI flag flip, and how `tests/cuda_runs.rs` runs the ICSBEP regression substrate under `--features cuda` without duplicating any scene setup code.

Where the GPU diverges in physics. A handful of features are not yet on the device:

- GPU survival biasing and Russian roulette. The CPU path runs both ($w_{\min} = 0.25$, $w_{\text{survive}} = 1.0$); the measured FOM gain on PWR is 4.5 ×. The GPU runs analog. This is variance-only, not a k-eff bias.
- GPU discrete S(α,β) inelastic (NJOY iwt modes 0 and 1). The OpenMC HDF5 distribution emits every TSL as `incoherent_inelastic` (continuous), which is exactly what the device sampler consumes; the upload path `panic!`s if the host hands it `InelasticDist::Discrete` so silent breakage is impossible.
- Per-cell `Mat3` rotation. `GpuRecursiveContext::build` errors out loudly if any cell sets `rotation = Some(...)`. No ICSBEP scene currently uses it, but the rejection is explicit rather than silent.

Process-wide shared context. `GpuTransportContext::shared()` returns an `Arc<GpuTransportContext>` from a process-wide `OnceLock`. First call pays the NVRTC compile + CUDA init cost (~30–150 ms on RTX A1000); every subsequent caller — across cases in an ICSBEP sweep, across PyO3 entry points, across rayon worker threads — gets the same Arc. This is the foundation the device-buffer caches sit on; a fresh context per case (the prior pattern) had empty caches on every entry. The shared context is also what makes the `Arc::as_ptr`-keyed caches work across cases: when `material_resolve::thermal_cache` on the host side returns the same `Arc<ThermalScatteringData>` to two different cases, the GPU side gets the same pointer and the cache hits.

3 Why we cache, and what it weighs

3.1 The structure of redundant work

A continuous-energy Monte Carlo case loads $O(20)$ nuclides at $O(10^5)$ energy points per nuclide; each nuclide carries multiple reaction-channel SVD bases / pointwise tables, an elastic angular distribution, fission outgoing-energy distributions, optional URR probability tables, and (for moderators) a separately-loaded S(α,β) thermal-scattering library with $O(10^2)$ incident energies, $O(10^4)$ outgoing-energy bins, and $O(10^5)$ angle bins per temperature. The host parses these from HDF5; the device wants them in flat-pack form ready for kernel pointer arithmetic. Both layers cost real time.

An ICSBEP corpus sweep does not load N distinct nuclides; it loads the same ~25 nuclides over and over. Every PWR pin cell uses H-1, O-16, U-235, U-238, Zr-90 through Zr-96; every Be-reflected fast-metal case uses Be-9 plus its `c_Be` TSL; every HEU solution case uses the same actinide block. A 375-case corpus at 5 seeds per case will, naively, parse `U235.h5` from disk **1 875 times** and HtoD-copy its SVD basis the same number of times.

We measured this on the `origin/main` state immediately before this paper's work landed. On a 73-case sub-sweep, `c_Be` was parsed from disk **125 times** and re-uploaded to the GPU the same number of times — roughly 8 GB of redundant HtoD copies for one TSL file alone.

3.2 Host-side: `thermal_cache` + `TieredStore`

The fix on the host side is straightforward and has been in the engine for a while:

- `material_resolve::thermal_cache` is a process-wide `RwLock<HashMap<PathBuf, Arc<ThermalScatteringData>>>`. Keyed on the canonical absolute path of the TSL HDF5 file. First parse populates the entry; every subsequent `resolve_materials` call for the same TSL hands back an `Arc::clone`. No size cap — TSL inventory per data dir is bounded ($\leq$ 20 distinct files) and the bytes are immutable.
- `transport::nuclide_cache::TieredStore` is the analogous structure for nuclide kernels, with an L1 in-memory tier (`Arc<NuclideKernels>`-keyed) and an eviction policy combining LFU and recency (`LfuEntries::evict_to_budget`).

The host caches use `Arc::as_ptr` (or canonical paths) as identity keys. The `Arc::as_ptr` approach matters because the underlying `NuclideKernels` or `ThermalScatteringData` structs are large ($O(50)$-MB) and content-hashing them every cache lookup would defeat the cache's purpose; pointer identity is a safe proxy for content identity *provided* every consumer goes through the same upstream loader. The loader's job is to return the same `Arc` for the same logical request — a contract we enforce by funnelling every consumer through `TieredStore::get_or_load` or `thermal_cache::load_or_get`.

3.3 Device-side: `per_nuclide_cache` and `sab_buffer_cache`

The device side needs a parallel cache structure because Arc pointer identity does not give you the device buffer; it gives you the host data the device buffer was once derived from. The first call has to upload; subsequent calls have to find the cached upload again. Our pattern, identical between the two caches:

1. **Key** is the `Arc::as_ptr` of the host `NuclideKernels` or `ThermalScatteringData`, cast to `usize`, plus any structural parameters the kernel ABI bakes in: SVD rank for nuclides; `n_nuc` (sizes the per-nuclide lookup table) plus the full slot list for SAB.
2. **Value** is an `Arc<GpuNuclideData>` (or `Arc<GpuSabData>`). Returning an Arc means a cache hit is one atomic refcount bump — no device traffic and no host-side rebuild of flat-pack arrays.
3. **Eviction** is LFU-with-recency from `nuclide_cache::eviction::evict_to_budget`. Both caches share the same `bundle_cache_budget` (75 % of detected VRAM, env-var overridable as `OPEN_RUST_MC_GPU_BUNDLE_CACHE_BYTES` or `_FRACTION`). The map / adapter is per-cache so an eviction in one does not perturb the other.
4. **Race handling** is the usual "check, compute outside the lock, then re-check on insert" three-phase protocol. If two threads race the same key during a sweep preload, the loser drops its upload and clones the winner's `Arc`. Both buffers being byte-identical (same upstream Arc, same upload pipeline) means the dropped upload is wasted work, not wasted correctness.

3.4 Order-sensitivity in the SAB key

S(α,β) takes one extra design decision the per-nuclide cache did not need. The `upload_sab_data_multi` routine packs N TSLs into a single flat layout, with per-slot offset arrays (`slot_inc_e_off`, `slot_eout_table_off`, ...) that index into the concatenated buffers. The slot order is the iteration order of the input slice. Two callers asking for the same logical slots in different orders produce different per-slot offset arrays, which means different cached device buffers.

We make the cache key order-sensitive: it stores `Vec<(arc_ptr, nuc_idx)>` in the upload order, not a sorted set. Two callers can construct the slot list however they like, but a sweep that consistently iterates `provider.thermal` in slot-index order (which is what every production caller does) gets clean cache hits. A future caller that wants order-independence can sort upstream of the call site.

3.5 What it weighs (measured)

We ran the smoke-verification sweep on heu-comp-inter-003_case-1 with 2 seeds, 30 batches, 5 000 particles, ENDF/B-VIII.1 nuclear data, on the dev-box RTX A1000 (4 GB), and snapshotted the GPU caches before and after the case via the new `--debug-metrics` path (§7).

phase	before	after
`sab_cache_entries`	0	1
`sab_cache_hits`	0	1
`nuc_cache_entries`	0	19
`nuc_cache_hits`	0	19
`nuc_cache_bytes`	0	2,468,534,728 B
`sab_cache_bytes`	0	300 B
`bundle_cache_budget`	3,220,881,408 B	(same)

Reading: 2.47 GB of per-nuclide device buffers cached — 19 distinct nuclides for one HEU-COMP-INTER case, under the 3.0 GB budget. The 19 hits across one case correspond to one hit per nuclide from the second seed; without the cache, the second seed would have re-uploaded all 19 nuclide payloads. The `sab_cache` weighs only 300 B for this case because the case's TSL binding is narrow; a Godiva or pure-Be case with a heavily-bound `c_Be` TSL pushes that into the tens of MB.

3.6 Why we did not yet add `material_data` caching

The material-composition upload (`upload_material_data`: per-material nuclide indices, atom densities, AWR table, $\bar{\nu}$ constants) is the third production GPU upload. It also rebuilds on every call. We did not cache it. The payload is in the low hundreds of KB for a realistic ICSBEP case, roughly three orders of magnitude smaller than the SAB payload it sits next to. The wasted bandwidth across a sweep is dominated by the SAB payload by the same factor. We chose to leave the engineering surface smaller until profile data shows the material-data path is non-trivial.

4 SM occupancy and register pressure

4.1 The `nuc_t[MAX_NUC_PER_MAT]` stack array

The hottest CUDA kernel, `trace_and_sample`, walks each particle to its next interaction. The macroscopic cross section at the particle's energy is the sum of every microscopic cross section in the material, weighted by atom density:

$$\Sigma_t(E) = \sum_{i \in \text{material}} N_i \, \sigma_t^{(i)}(E).$$

The kernel evaluates every term as it computes the sum, and *also* retains each $N_i \sigma_t^{(i)}(E)$ in a per-thread stack array `nuc_t[MAX_NUC_PER_MAT]` because the reaction-selection sampler that runs after the sum needs to draw a nuclide proportional to the per-nuclide contribution. The straightforward implementation stores all of those contributions, then samples on a second pass.

`MAX_NUC_PER_MAT` is the single source-of-truth constant `crate::MAX_NUCLIDES_PER_MATERIAL = 128` that we bumped from 32 last year to accommodate HMF-069 (69 nuclides) and Pu solution cases (67). The CPU side imports the constant; the GPU side receives it via NVRTC `-DMAX_NUC_PER_MAT=N` so `transport.cu` sizes the stack array correctly. The header `#errors` out if NVRTC forgets the flag — silent breakage was an early hazard.

The cost: $128 \times 8\,\text{B} = 1\,\text{KB}$ of per-thread stack on Ampere. On sm_86 each SM has 65 536 32-bit registers total; with 1 KB of stack array allocated, plus the other live registers in the trace-and-sample body (energy bin index, sampled cumulative, RNG state, particle vector, surface / cell descent stacks), the compiler often promotes the array into stack-frame memory (local memory in CUDA terminology), which serializes to L1 / L2 instead of staying in registers.

We considered streaming the per-nuclide loop, computing each $N_i \sigma_t^{(i)}(E)$ and immediately folding it into a running cumulative without retaining `nuc_t[]`, then evaluating the cross sections a second time during reaction selection. This doubles the XS evaluation cost but eliminates the register spill. We did not commit to either branch: the trade-off depends heavily on whether the per-nuclide loop is bound on register pressure (favouring streaming) or on XS evaluation cost (favouring caching). Both are documented in the source comments as future work and are part of the agenda when we get a representative sweep on H100-class hardware where the larger register file changes the trade-off.

4.2 Compute-capability-specific tuning constants

A handful of magic numbers in the kernel-launch heuristics depend on the device:

- **Blocks per SM.** sm_86 (Ampere RTX A1000 / 3080) and sm_89 (Ada L40 / RTX 4090) limit to 16 blocks per SM; sm_90+ (Hopper, Blackwell) raises this to 32. Our `recommend_refill_factor` function reads the compute capability via cudarc's `CU_DEVICE_ATTRIBUTE_COMPUTE_CAPABILITY_MAJOR` and routes both branches.
- **Register file size.** sm_75 (Turing T4), sm_86, sm_90 all carry 65 536 registers per SM. We use a conservative ceiling that does not need to fork on this constant.
- **atomicAdd(double*, double) availability.** Introduced in CC 6.0 (Pascal GP100). We reject earlier compute capabilities explicitly at NVRTC compile time with a named error message — silent fallback would have meant the kernel compiles but produces wrong tallies on any device that synthesises the atomic via lock-cmpxchg.

4.3 What we believed vs. what we measured

Two priors we revised after measurement:

The SM-coverage threshold for refill is geometry-dependent. Our first refill-pool experiment assumed that any time the batch's particle population dropped below the "SM count×max threads per SM" threshold, refilling would help. Measurement (§5) showed the threshold is not a single number: under-occupied initial populations don't benefit (the problem is at step 0, not in the tail), and over-occupied initial populations also don't benefit (no SM headroom for refilled work). Refill wins in the mid-curve where the initial population fills the SMs but the population decays over ~5–10 events. The right tuning constant is the saturation *knee* of the device, not the SM count.

Memory-bandwidth limits do not show up where we expected. We expected SVD reconstruction to be compute-bound (one rank-k FMA chain per lookup). Profiling showed the bottleneck is not the FMA but the divergent loads — 32 threads in a warp each indexing into per-nuclide `basis` and `coeffs` arrays at different offsets and energy bins, a pattern the Monte-Carlo XS literature catalogues as the classical memory-bottleneck regime [1]. The basis and coefficient arrays are cache-blocked (per-nuclide pointer arrays since `gpu_per_nuclide.rs`'s pointer-table work), but the *access pattern* across a warp is the limiter, not the arrays' aggregate bandwidth.

5 Refill pool and saturation

5.1 The problem refill solves

Within an active batch the GPU pipeline starts with N particles and walks them forward in lock-step. Particles die at different times — some leak immediately, some live for $O(10)$ collisions. The kernel keeps a per-particle "alive" flag that each step gates on; when many particles die, the SM lanes allocated to them sit idle for the rest of the batch. Population decay over a typical Godiva history halves the live count every ~3–5 steps, so by the tail of the batch the SMs are running at a small fraction of nominal occupancy.

The fix from Tramm et al. [2] (and earlier PHYSOR 2022 "Toward Portable GPU Acceleration of OpenMC") is a refill pool: oversize the source bank by a factor f, and refill dead slots from the overflow between event-pipeline steps. The kernel self-gates on `alive[tid] == 0` per particle and picks a refill from a device-side counter.

5.2 Saturation curve on RTX 3080

Before instrumenting the refill, we measured the device's saturation curve on HMF-001 Godiva (3 nuclides, fast spectrum), 80 batches / 60 active, seed 1, rank 15 (`outputs/saturation_*.csv`):

particles	sim wall (s)	μs / p	hist / s
10 000	6.27	10.45	~95 k
30 000	7.50	4.17	~240 k
50 000	8.83	2.94	~340 k
100 000	11.02	1.84	~545 k
200 000	15.78	1.31	~760 k
500 000	28.44	0.95	~1 055 k
1 000 000	49.87	0.83	~1 203 k

Knee at 500 k–1 M particles per batch. Per-doubling parallel efficiency: 84 % from 10 → 30 k, drops to 57 % between 500 k→1 M. The default `icsbep_run.py particles=5000` runs the 3080 at ~ 8 % of capability; bumping to 1 M is a free 14 × throughput win on the same hardware.

5.3 Measured refill gain on mid-curve workload

We then validated the refill instrumentation on HMF-008 (21 nuclides, fast metal), 250 k particles per batch, 80 batches / 20 inactive (`outputs/hmf008_refill_*.csv`):

metric	refill off	refill 2×
sim wall (s)	48.67	48.23
collisions	41.9 M	83.8 M
fissions	5.73 M	11.47 M
μs / collision	1.162	0.575
k_{calc}	0.99456	0.99485
σ_k	0.00040	0.00019
Δk vs. ref	−124 pcm	−95 pcm
status	PASS	PASS

Refill at $f = 2$ doubled the collisions in the same wall time, tightened σ_k by 2.1 × (vs. the $\sqrt{2} \approx 1.41$ × naive prediction — the inactive-batch source convergence also benefits), and dropped μs/collision by 50.5 %. The +29 pcm shift in k_{calc} is MC noise; both rows pass the ICSBEP envelope.

5.4 The three refill regimes

The above is one point on the curve. The full pattern, from sweeps at varying initial particle counts, is:

Under-occupied (Particles $\lesssim$ 5 000 on RTX A1000, $\lesssim$ 35 000 on RTX 3080). The initial population does not fill the SMs at step zero. Refill cannot help — the problem is at step zero, not in the tail. Refill adds wall time with no payoff. A1000 smoke at 5 000 particles measured 0.987 × throughput.

Mid-curve (Initial population fills the SMs at step zero but decays over ~5–10 events — typical Godiva / HMF-class fast-metal). Refill closes the tail gap. Measured 1.5 ×–2.0 × throughput at $f = 2$.

Saturated (Initial population so large that even after 80 % die-off the surviving alive count still fills the SMs — RTX 3080 at ≥ 1 M particles per batch per the saturation table above). Refill adds memcpy / kernel-launch overhead with no SM headroom left to fill. Expected gain shrinks to ~5–10 %.

5.5 The auto-refill recommender

Picking f by hand requires the operator to know the device's saturation knee for the current geometry. We added a device-attribute-driven recommender, `recommend_refill_factor`, that queries SM count, max threads per SM, and the per-thread register count of the trace-and-sample kernel, then returns the factor that fills available SM lanes given the particle count the sweep is about to launch. It returns `None` when the workload is either under-occupied or already at saturation — "no recommendation, accept analog". The sweep harness prints `GPU auto-refill: picked factor=1.50` or `GPU auto-refill: no recommendation for particles_per_batch=500000` so the operator sees the choice without setting a flag.

6 Adaptive architecture targeting

The original NVRTC compile sites hardcoded `arch: Some("sm_86")` — the Ampere compute capability of the development RTX A1000 / RTX 3080. This made the binary device-specific. Running on a cluster H100 (sm_90), an L40 (sm_89), or a Blackwell RTX 5090 (sm_120) required either an explicit rebuild for that target or accepting that NVRTC would back-target sm_86 PTX and emit a runtime warning about deprecated PTX features.

6.1 Implementation

We added `gpu_recursive::device_nvrtc_arch` that queries the device's CC at runtime via the existing `CU_DEVICE_ATTRIBUTE_COMPUTE_CAPABILITY_{MAJOR,MINOR}` attribute pattern, builds the `sm_{major}{minor}` string, and passes it to both NVRTC compile sites (`GpuRecursiveContext::new` for recursive kernels, `GpuTransportContext::new` for flat transport).

The minimum supported compute capability is CC 6.0 / sm_60 (Pascal). Below that the helper returns an explicit error naming the detected capability and explaining that `atomicAdd(double*, double)` is unavailable. Falling through silently would compile a kernel that synthesises the atomic via lock-cmpxchg and corrupts the spectrum-hardening tallies under contention.

The helper is `pub` (not `pub(crate)`) so binding crates and external benchmarks can target the detected device without re-implementing the query. `GpuTransportContext` caches the arch string at construction time, exposed via `nvrtc_arch()`, so observability paths can surface what arch the kernels were actually compiled against.

6.2 Cost

One bounded ~6-byte `Box::leak` per process per context (NVRTC's `CompileOptions.arch` is `Option<&'static str>`; the leaked string lives for the process lifetime, which matches when the GPU context is released anyway). The `OnceLock`-based shared context means this leak happens at most once per process even across ICSBEP sweep cases.

6.3 What this unblocks

A cluster operator running the engine across a heterogeneous fleet can deploy one binary; each node compiles its own PTX at startup against its own hardware. The same applies to cloud GPUs that come and go: an H100 spot instance does not need a custom build, just the binary and the data.

7 Observability via `--debug-metrics`

7.1 The problem we kept solving by hand

While developing the per-nuclide cache and (later) the SAB cache, we repeatedly wanted to answer: *did the cache actually fire this case?* The engine prints diagnostic lines for cache misses (per-slot SAB upload prints from `pack_sab_elastic`, nuclide-load progress lines from `NuclideLibrary::load_or_get`), so a cache hit shows up as the *absence* of expected log lines.

This works on the CPU path because the engine's stdout flows straight to the operator's terminal. On the Python sweep path it does not: `icsbep_sweep.py` captures the engine's stdout internally and emits only per-case summary lines. The diagnostic lines from inside the cache miss never reach the log file, and the operator cannot tell from the log whether the cache fired or whether the engine just happens to print less per call.

7.2 The fix

We exposed three accessors on the shared GPU context:

- `per_nuclide_cache_stats()` returns `(n_entries, total_bytes, total_hits)`. `total_hits` is the sum of per-entry `EvictionStats::hits`, which counts how often the cache served from an existing entry rather than uploading.
- `sab_buffer_cache_stats()` returns the same triple for the SAB cache.
- `nvrtc_arch()` returns the `sm_{major}{minor}` string the kernels were compiled against, cached at `new()`.

These flow to Python via a new module-level `gpu_debug_metrics()` PyO3 function that returns a dict:

```
{
  "gpu_arch": "sm_86",
  "sab_cache_entries": <int>,
  "sab_cache_bytes":   <int>,
  "sab_cache_hits":    <int>,
  "nuc_cache_entries": <int>,
  "nuc_cache_bytes":   <int>,
  "nuc_cache_hits":    <int>,
  "bundle_cache_budget_bytes": <int>,
}
```

The CPU-only build of the same function returns `None` so Python callers do not need to feature-gate their imports.

The sweep driver gets a single new flag, `--debug-metrics PATH`, that opens the file append and writes one JSON line per snapshot: an `init` line before the per-case loop (empty caches, current device arch, runner type, total case count) and an `after_case` line per case (case name, wall time, status, k_{calc}, σ_k, and the full metrics dict). The per-case delta in `*_hits` between consecutive lines gives the number of cache hits that case took.

7.3 Smoke verification

To convince ourselves the new `sab_buffer_cache` actually fires at runtime, we ran the 2-seed smoke on heu-comp-inter-003_case-1 (the same case the natural-element migration ran against in the data-library transition) with `--debug-metrics outputs/sab_cache_smoke.debug.jsonl`. The init line showed all-zero cache state. The `after_case` line showed `sab_cache_entries=1`, `sab_cache_hits=1`, `nuc_cache_entries=19`, `nuc_cache_hits=19`.

The pattern (`entries = hits = N`) is the textbook "N first-seed misses + N second-seed hits" trace — the first seed populated the cache, the second seed hit every entry. Without the cache the second seed would have re-uploaded all 19 nuclide payloads (2.47 GB) and the one SAB slot. Without `--debug-metrics` we could not have proven this from the sweep log alone.

7.4 Beyond cache validation

The same path is the foundation for any future GPU-side telemetry we want surfaced in sweeps:

- Per-case GPU memory peak, queried via `cudarc::driver::result::device::total_mem` + the kernel's `stream`'s allocator.
- Per-case kernel launch counts and µs-per-launch averages.
- Refill-pool factor selected by `recommend_refill_factor` (currently logged to stdout; future work to surface as a metric).

The PyO3 wrapper is a ~30-line addition per metric category; the cache-stat work proves the shape.

8 What we thought vs. what we measured

A consolidated honesty pass on this engine's GPU story. Each finding is paired with the prior we held and the measurement that revised it.

8.1 "GPU SVD wins because the reconstruction kernel is fast"

Prior: The SVD reconstruction microbench measures 8–13× per-XS-lookup speedup over a single-threaded CPU loop. We expected this to translate to a ~5–10× end-to-end GPU SVD speedup over CPU SVD for realistic transport.

Measurement (`outputs/method_comparison_2026-05-08.txt`): On Godiva, GPU SVD vs. 20-core rayon CPU SVD on a workstation Intel mobile: 0.77×. The GPU SVD path is 1.3× *slower* than the parallel CPU path on integrated transport. On the same problem against GPU pointwise table (apples to apples on the same backend): also 0.77×. The launch overhead + divergent memory access dominates the per-XS-lookup speedup at the problem scale where Godiva sits.

Lesson: Microbench speedups do not survive integration when (a) the kernel does O(microbench unit) arithmetic per the much-larger geometry-traversal + collision- sampling work envelope, and (b) the per-XS-lookup pattern is divergent within a warp.

8.2 "Recursive transport is 6.74× on GPU"

Prior: Documented in earlier session notes, propagated into prior STATUS.md headlines, cited in conversations as a general "GPU is faster than CPU" fact.

Measurement: The 6.74× number is real but scope-tagged. It comes from `const_xs_transport_persistent` — a constant-XS microbench that exercises the recursive geometry walker plus fission banking via `atomicAdd`, but skips all the SVD / table XS evaluation, all the angular distribution sampling, and the energy-distribution sampling. As a measurement of how fast *geometry traversal* runs on the GPU, it is honest. As a proxy for "how fast our full transport runs on GPU" it is misleading.

Lesson: Always carry the scope tag. `[micro]`, `[godiva]`, `[assembly]` are not decoration; they signal whether the number generalises. The audit in this paper's preparation revised every header table in `STATUS.md` and `README.md` to make this distinction explicit.

8.3 "Refill is 2× ICSBEP-wide"

Prior: After the first refill measurement at HMF-008 at 250 k particles showed a clean 2× throughput gain, we expected the same uplift across the sweep.

Measurement (§5): The refill gain is a function of where on the saturation curve the workload sits. At 5 000 particles on RTX A1000, throughput is 0.987× — refill adds overhead. At 250 k on RTX 3080 (mid-curve), 2.0×. At ≥1 M on RTX 3080 (saturated), the gain shrinks to single-digit percent. There is no universal refill factor; the auto-recommender that reads device attributes and routes by regime is the necessary engineering, not the refill kernel itself.

Lesson: Optimisations that exploit a specific occupancy regime need device-aware dispatch. Static defaults are either wrong or under-utilised on most cards.

8.4 "CAB H-in-H2O fixes the HEU-COMP-INTER benchmark family"

This finding is from the ENDF/B-VII.1 vs. VIII.1 transition that happened during this paper's preparation and is documented in `STATUS.md` and a recent commit. We include it here because the diagnostic process used the same observability-first approach.

Prior: VIII.1 ships the CAB H-in-H_2O thermal scattering re-evaluation. We expected this to pull the HEU-COMP-INTER-003 family closer to the handbook k=1.000 because "better thermal scattering data should improve thermal-spectrum benchmarks".

Measurement (7-case A/B, CPU runner, 5 seeds, 150 batches, 40 inactive, 100 k particles on RTX A1000 host): VIII.1 shifts *k upward* by +820 ± 153 pcm uniformly across the family. Cases that were marginally PASS under VII.1 became FAIL under VIII.1 (case-2, case-3 against the handbook); cases that were below 1.0 under VII.1 crossed zero into more comfortable PASS (case-5, -6, -7).

Root cause via h5py probe: VIII.1's CIELO-era U-235 re-evaluation raised σ_f at the intermediate-spectrum peak (100 eV) by +5.3 % and dropped σ_γ by −5.5 %, so the capture/fission ratio α dropped by −10.2 %. The benchmark family's flux peaks at exactly that energy. The CAB H-in-poly TSL change was not the dominant driver here — the H-in-CH_2 TSL was not even bound on the loaded geometry (the only TSL binding in this family is `c_Be`).

Lesson: When a benchmark family shifts under a library change, the right diagnostic is to probe the library directly (h5py at known-resonance energies) before assigning the shift to a specific physics mechanism. The intuitive hypothesis ("the TSL update fixed it") was specifically wrong about which TSL even applied.

8.5 "Cache machinery just needs the right key"

Prior: Implementing a cache is straightforward; the key design is the only interesting bit.

Measurement: The cache itself is straightforward, but *verifying* the cache fires at runtime turned out to be the harder problem. The engine's stdout was captured by the Python sweep driver and never reached the log file. Without `--debug-metrics`, an operator running an ICSBEP sweep has no way to falsify "the cache is working". We spent multiple sessions reading code and convincing ourselves the cache must be firing; the actual confirmation needed a ∼200-line observability path.

Lesson: Design the falsification path at the same time as the optimisation. Cache hits are an absence of work, and absences of work are hard to see without explicit telemetry.

9 Conclusion and future work

9.1 What landed

- Process-wide per-nuclide and S(α,β) device- buffer caches keyed on `Arc::as_ptr`, sharing one bundle-cache byte budget. Measured: 2.47 GB of per-nuclide buffers + N-MB SAB buffers cached after one case, 19 + 1 cache hits on the second seed of the same case.
- Refill pool (PHYSOR 2022 Optimization F) with device- attribute-driven auto-recommender. Measured saturation curve on RTX 3080 (5k→1M particles, 1.2 M h/s peak) and mid-curve refill gain (2.0 × histories at the same wall on HMF-008 at 250k particles).
- Adaptive NVRTC arch detection. One binary now targets RTX A1000 (sm_86), A100 (sm_80), H100 (sm_90), RTX 5090 (sm_120). CC ≥ 6.0 enforced; below that the helper returns a named error rather than silently compiling unsafe atomics.
- Structured observability via `--debug-metrics PATH` + new `gpu_debug_metrics()` PyO3 function. One JSON line per case captures cache entry counts, hit totals, byte usage, and the detected compute capability.

9.2 What is queued

- GPU survival biasing / Russian roulette. CPU FOM on PWR is 4.5 ×. The GPU runs analog. Variance- only — not a k_{eff} bias — but a known throughput gap.
- GPU material-payload cache. Pattern identical to the SAB cache; payload is ∼3 orders of magnitude smaller per call. Defer until profile data shows it is worth the code surface.
- Energy-bin sort within reaction class (PHYSOR 2022 Optimization G). The reaction-sort already exists; a secondary per-class energy sort would improve XS-lookup memory locality at the cost of one small device-side sort per step. Paper measured 1.3 × on A100; we have not measured on our hardware yet.
- `nuc_t[]` streaming. Eliminating the per-thread stack array trades an extra XS evaluation pass for improved register occupancy. Worth re-evaluating on H100-class hardware where the larger register file changes the trade-off.
- HexLattice GPU runtime parity sweep. The device-side `gr_hex_*` functions are wired but a full large-volume CPU vs. GPU parity test (equivalent to the existing CPU `hex_lattice_descent_and_trace_smoke`) is pending.

9.3 Where this leaves us

The honest reading is that most of these problems were already defined and known in the literature. The XS-lookup memory- bandwidth regime is in Tramm & Siegel 2014 [1]; the GPU saturation curve and refill-pool optimisation are in the PHYSOR 2022 / EPJ 2024 work from the OpenMC group [2]; the per-warp shared-memory (n, E_{idx}) cache idea was a published optimisation that the SVD-paper companion text already documents as a falsified hypothesis on this geometry. We did not discover new performance mechanisms in this engine's CUDA port. We took our time hitting known ones.

What this paper documents is the implementation pace and the order in which the lessons arrived: where the per-XS-lookup microbench misled us into expecting an integration win, how long it took to notice that the SAB upload was the missing peer of the per-nuclide cache, why "GPU is 6.74 ×" was carried across sessions without its scope tag, how many sessions were spent reading code to convince ourselves the cache fired before we added the observability (§7). The CPU backend on a 20-thread laptop beats GPU SVD on

Godiva and is competitive through PWR; the GPU wins materialise at problem scales where rayon's cache contention dominates (large assemblies, multi-case ICSBEP sweeps, photon shielding with 10^6+ histories), which is also what the source literature predicts.

References

[1] John R. Tramm and Andrew R. Siegel. Memory bottlenecks and memory contention in multi-core Monte Carlo transport codes. *Annals of Nuclear Energy*, 82: 195–202, 2015. doi: 10.1016/j.anucene.2014.09.028.

[2] John Tramm, Paul Romano, Patrick Shriwise, Amanda Lund, Johannes Doerfert, Peter Steinbrecher, Andrew Siegel, and Gavin Ridley. Performance portable Monte Carlo particle transport on Intel, NVIDIA, and AMD GPUs. *EPJ Web of Conferences*, 302:04010, 2024.